Weekly Progress

ExpenseEase - Shared Expense Management System | Implementation status and evidence

ExpenseEase follows an **iterative and incremental SDLC**: establish requirements and architecture, implement a small end-to-end increment, validate it through tests and user-facing evidence, then refine the next workflow. This view focuses on engineering progress and implementation evidence.

4 / 4	179	PASS	17
increments tracked as complete	automated tests passed	lint and production build	UI screenshots captured

Increment-by-increment progress

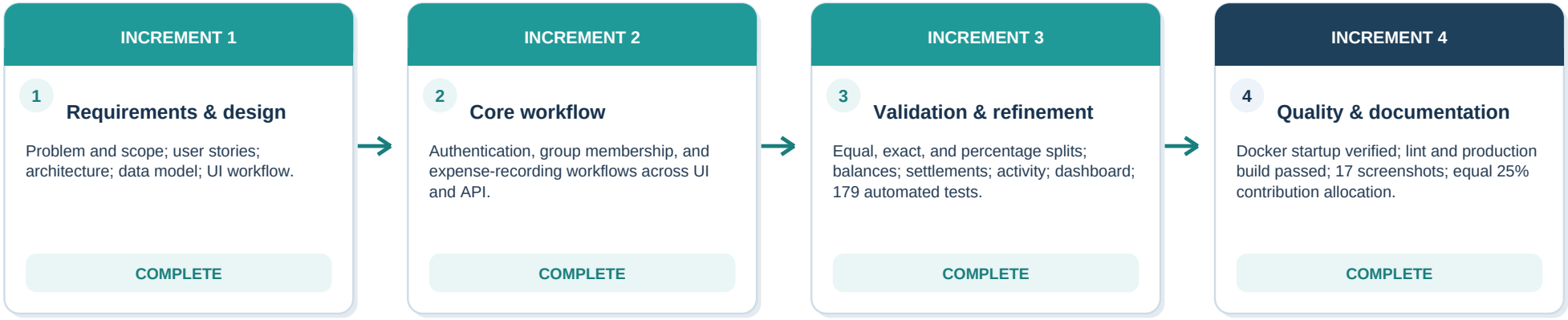
LIFECYCLE STAGE	GOAL / SCOPE	COMPLETED EVIDENCE	ENGINEERING STATUS
Increment 1 **Requirements & design**	Define the problem, scope, user stories, architecture, data model, and UI workflow.	Requirements and design artifacts prepared; responsibilities divided equally across all four members.	**Complete** No technical blocker is recorded in this stage.
Increment 2 **Core workflow**	Deliver registration/authentication, group membership, and expense recording.	Frontend and backend authentication, group, membership, and expense workflows implemented.	**Complete** Core end-to-end workflows implemented.
Increment 3 **Validation & refinement**	Add split validation, balances, settlements, activity history, dashboard, and tests.	Equal, exact, and percentage splits; settlements; activity; dashboard; 179 automated tests completed.	**Complete** Test suite passed: 157 server + 22 client tests.
Increment 4 **Quality & documentation**	Verify container startup, quality checks, test evidence, and project documentation.	Docker startup, lint, production build, 17 screenshots, and equal 25% contribution allocation recorded.	**Complete** Lint and production build passed.

Lifecycle approach	Requirements and design lead into a working increment; each increment is integrated and tested before the next workflow is added. The PERT-style dependency chart on the next page summarizes this sequence.

PERT / Dependency Chart

ExpenseEase - Shared Expense Management System | Incremental lifecycle and dependency path

The diagram shows the dependency order of the four increments. The source progress record does not provide start dates or task durations, so this is a dependency-focused PERT-style view rather than a calendar-based Gantt schedule.



Dependency path: I1 -> I2 -> I3 -> I4

Verification evidence

TESTS	STATIC QUALITY	RUNTIME / CONTAINER	PROJECT ARTIFACTS
179 automated tests passed: 157 server and 22 client.	Lint and production build completed successfully.	Docker startup verified.	17 screenshots and the equal 25% team contribution record are documented.

Dependency path: Requirements & design → core workflow → validation and refinement → quality and documentation.